

Authentication & Product **CRUD APIs**

JWT Access + Refresh Tokens, Product CRUD APIs & Frontend with Express Validator

● Node.js

● Express

● MongoDB

● React

● express-validator

Objective — Build a secure REST API for a small e-commerce platform, along with a simple frontend to consume it. Implement a complete JWT auth flow, CRUD APIs for the Product resource with express-validator, and a frontend built with genuine understanding.

TASK	WHAT YOU BUILD	KEY REQUIREMENT
1 Authentication APIs	Register, login, refresh, logout, me	JWT access + refresh tokens
2 Product CRUD APIs	Create, read, update, delete products	Protected write routes
3 Validation	express-validator on all inputs	Field-level 400 errors
4 Frontend	Auth pages + product UI	Must understand every line
5 Deliverables	README + GitHub repo + live link	Single repository

1 Authentication APIs

TASK 1

Implement a complete auth module with the following endpoints.

METHOD	ENDPOINT	ACCESS	DESCRIPTION
POST	/api/auth/register	Public	Create a new user account
POST	/api/auth/login	Public	Authenticate user, issue access + refresh tokens
POST	/api/auth/refresh-token	Public*	Issue a new access token
POST	/api/auth/logout	Authenticated	Invalidate the refresh token
GET	/api/auth/me	Authenticated	Return the logged-in user's profile

* requires a valid refresh token

2.1 — Register API requirements

- ▶ Accept `name`, `email`, `password`, `confirmPassword`
- ▶ Hash the password with `bcrypt` (min. 10 salt rounds) before saving — never store plain text
- ▶ Reject duplicate emails with a clear 409 error
- ▶ Return the created user (without password) — do not return tokens on register

2.2 — Login API requirements

- ▶ Accept `email`, `password`
- ▶ Verify password with `bcrypt.compare`; return 401 on mismatch (generic message, don't reveal which field is wrong)
- ▶ On success, generate an **Access Token** — short-lived (10–15 min), signed with `ACCESS_TOKEN_SECRET`, sent in the JSON response body
- ▶ Generate a **Refresh Token** — long-lived (7 days), signed with `REFRESH_TOKEN_SECRET`, sent as an `httpOnly`, secure cookie (or in body if you document why)
- ▶ Persist the refresh token (or its hash) in the database against the user, so it can be revoked on logout

2.3 — Access & Refresh Token logic

- ▶ Write an `authenticate` middleware that reads the Bearer access token from the `Authorization` header, verifies it, and attaches `req.user`
- ▶ `/api/auth/refresh-token` reads the refresh token, verifies it against the DB record, and issues a brand-new access token (and optionally rotates the refresh token)
- ▶ Expired/invalid/reused refresh tokens must return 401/403 and force re-login
- ▶ `/api/auth/logout` must delete/invalidate the stored refresh token and clear the cookie

Security checklist — passwords hashed (never logged/returned), JWT secrets in `.env`, refresh tokens stored server-side for revocation, `httpOnly` cookies for refresh tokens, rate-limit login (optional bonus).

2

Product CRUD APIs

TASK 2

Design and implement full CRUD (Create, Read, Update, Delete) APIs for the Product resource of the e-commerce site. Write routes (POST/PUT/DELETE) must be protected by the `authenticate` middleware.

METHOD	ENDPOINT	ACCESS	DESCRIPTION
POST	<code>/api/products</code>	Authenticated	Create a new product
GET	<code>/api/products</code>	Public	List all products (pagination optional)
GET	<code>/api/products/:id</code>	Public	Get a single product by ID
PUT	<code>/api/products/:id</code>	Authenticated	Update a product
DELETE	<code>/api/products/:id</code>	Authenticated	Delete a product

Note — Every resource must be scoped and validated correctly — a product should only be updated or deleted after its `:id` is confirmed to exist.

3 Validation with express-validator

TASK 3

All routes that accept body, params, or query input — for both the authentication APIs and the product CRUD APIs — must implement request validation using **express-validator**. Invalid requests should be rejected with a clear 400 response before reaching the controller logic.

- ▶ **Auth inputs** — name, email format, password strength, matching confirm-password.
- ▶ **Product inputs** — required fields, correct data types (e.g. price and stock as numbers).
- ▶ **Route params** — validate `:id` before querying the database.
- ▶ **Error format** — return field-level error messages so the API consumer knows exactly what failed.

4 Frontend — Mandatory, But Build It Yourself

TASK 4

Students must also build a simple **frontend** to consume these APIs (register/login pages, a product listing, add/edit/delete product forms, etc.). Any stack is fine — plain HTML/CSS/JS or a framework like React.

You are allowed to take AI's help while building the frontend — but the goal is to understand and learn, not to blindly copy-paste whatever AI generates. If something doesn't work or you don't know how to do it, use AI to learn the concept and the logic, then write and build it yourself in your own understanding.

You must be able to explain **every part of your code** — what each function does, how data flows from the frontend to the API and back, why a particular approach was used, etc. If you were asked to explain any particular block of code or its working then definitely you must be able to explain it.

5 Deliverables & Submission

TASK 5

- ▶ Include a `README.md` file in the repository explaining the project, setup steps, and the list of available API endpoints.
- ▶ Push both backend and frontend code to the same repository.
- ▶ Submit the **GitHub repository link** and the Project **Live Link** — no other submission format is required.

Review note — Backend correctness, security, validation, and genuine understanding of both backend and frontend code will all be checked during review.